

67. Add Binary Easy

Acceptance: 56.7% | [LeetCode](#)

Given two binary strings `a` and `b`, return *their sum as a binary string*.

Example 1:

```
**Input:** a = "11", b = "1"
**Output:** "100"
```

Example 2:

```
**Input:** a = "1010", b = "1011"
**Output:** "10101"
```

Constraints:

- `1 <= a.length, b.length <= 104`
- `a` and `b` consist only of `'0'` or `'1'` characters.
- Each string does not contain leading zeros except for the zero itself.

Claude Opus Solution

Add Binary - Solution Explanation

1. Intuition

The key insight is to **mimic how we add numbers by hand** (grade-school addition), but in base 2 instead of base 10.

When adding two binary numbers: - We process digits **from right to left** (least significant bit first). - At each position, we sum the two bits plus any **carry** from the previous position. - In binary, the sum at each position can be `0`, `1`, `2`, or `3`: - The **output bit** is `sum % 2` (the remainder) - The **new carry** is `sum // 2` (the quotient)

This is the universal pattern for "add two numbers represented as strings/arrays" problems.

2. Approach (Pseudocode)

1. Initialize: `result = ""`, `carry = 0`
2. Set pointers `i`, `j` to the last index of `a` and `b`
3. While `i >= 0 OR j >= 0 OR carry != 0`:
 - a. `total = carry`
 - b. If `i >= 0`: add bit `a[i]`, decrement `i`
 - c. If `j >= 0`: add bit `b[j]`, decrement `j`
 - d. `result_bit = total % 2` -> prepend to `result`
 - e. `carry = total // 2`
4. Return `result`

The trick of including `carry != 0` in the loop condition lets us handle the final carry *inside* the loop, avoiding a separate check afterward.

3. Python Solution

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        # Build result digits in a list (efficient appends), reverse at end.
        result = []
        carry = 0

        # Two pointers starting at the LEAST significant bit (rightmost char).
        i, j = len(a) - 1, len(b) - 1

        # Continue while there are bits left in EITHER string,
        # or there's still a carry to place. Including `carry` in the
        # condition handles the leftmost carry without extra post-processing.
        while i >= 0 or j >= 0 or carry:
            total = carry # Start with carry from the previous column.

            # Add bit from `a` if we haven't run past its start.
            if i >= 0:
                total += int(a[i]) # '0'/'1' -> 0/1
                i -= 1
```

```

# Add bit from `b` if we haven't run past its start.
if j >= 0:
    total += int(b[j])
    j -= 1

# The current bit of the answer is the remainder (0 or 1).
result.append(str(total % 2))

# The carry passed to the next (more significant) column.
carry = total // 2

# We appended least-significant bits first, so reverse to get correct order.
# ''.join is O(n) and faster than repeated string concatenation.
return ''.join(reversed(result))

```

Why a list instead of `r = z + r`?

The reference solutions use `r = z + r`, which creates a **new string each iteration** $\rightarrow O(n^2)$ total in the worst case. Using a list with `append` and a final `join` keeps it linear $O(n)$.

4. Complexity Analysis

Let `n = len(a)` and `m = len(b)`.

- **Time Complexity:** $O(\max(n, m))$
- We process each bit exactly once. The final `reversed + join` is also linear in the output length, which is $\max(n, m) + 1$ at most.
- **Space Complexity:** $O(\max(n, m))$
- The `result` list (and the returned string) hold the answer, whose length is at most $\max(n, m) + 1$. This is required output space, not auxiliary overhead.

5. Alternative Approaches

A. Built-in conversion (concise but "cheats")

```

class Solution:
    def addBinary(self, a: str, b: str) -> str:
        # int(x, 2) parses a binary string; bin() formats back to binary.
        return bin(int(a, 2) + int(b, 2))[2:] # [2:] strips the '0b' prefix

```

- **Pro:** One line, very readable.
- **Con:** Hides the algorithm; in languages without big-integer support, large inputs (up to 10^4 digits) would overflow. Python handles big ints natively, so this works but defeats the learning purpose.

B. Bit Manipulation (XOR + AND + shift)

Treat addition as: - sum without carry = `a XOR b` - carry = `(a AND b) << 1` - Repeat until carry is 0.

This is elegant for integers but awkward for arbitrary-length **strings**, so it's mainly of theoretical interest here.

Key Takeaways

- **Pattern:** Right-to-left digit-by-digit addition with carry — applies to *Add Strings*, *Add Two Numbers* (linked list), and multiplication problems.
- **Optimization tip:** Avoid string concatenation in loops; use a list + `join`.
- **Clean loop trick:** Add `carry` to the `while` condition to elegantly handle the final overflow bit.

Codex Solution

1. Intuition

Binary addition works exactly like decimal addition, but the base is 2.

For each bit position from right to left:

```
sum = bit_from_a + bit_from_b + carry
```

Then:

```
result_bit = sum % 2
carry      = sum // 2
```

Example:

```
  1010
+ 1011
-----
```

Start from the rightmost bits:

```
0 + 1 + 0 = 1 -> write 1, carry 0
1 + 1 + 0 = 2 -> write 0, carry 1
0 + 0 + 1 = 1 -> write 1, carry 0
1 + 1 + 0 = 2 -> write 0, carry 1
final carry = 1
```

So the answer is:

```
10101
```

The key pattern is:

Process both strings from right to left, maintaining a carry.

2. Approach

Pseudocode

```
function addBinary(a, b):
    i = last index of a
    j = last index of b
    carry = 0
    result = empty list

    while i >= 0 OR j >= 0 OR carry != 0:
        total = carry

        if i >= 0:
            total += integer value of a[i]
            i -= 1

        if j >= 0:
            total += integer value of b[j]
            j -= 1

        current_bit = total % 2
        carry = total // 2

        append current_bit to result

    reverse result
    return result as string
```

Why append and reverse?

Because we compute bits from least significant to most significant, meaning from right to left. So the result is built backwards and must be reversed at the end.

3. Python Solution

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        # Start from the rightmost character of both strings
        i = len(a) - 1
        j = len(b) - 1

        # Carry from the previous bit addition
```

```

carry = 0

# Store result bits in reverse order.
# Using a list is more efficient than repeatedly prepending to a string.
result = []

# Continue while there are bits left in either string,
# or there is still a carry to process.
while i >= 0 or j >= 0 or carry:
    total = carry

    # Add current bit from a if available
    if i >= 0:
        total += int(a[i])
        i -= 1

    # Add current bit from b if available
    if j >= 0:
        total += int(b[j])
        j -= 1

    # In binary:
    # total % 2 gives the bit to write at this position
    # total // 2 gives the carry for the next position
    result.append(str(total % 2))
    carry = total // 2

# The result was built from right to left,
# so reverse it before returning.
return ''.join(reversed(result))

```

4. Complexity Analysis

Let:

```

n = len(a)
m = len(b)

```

Time Complexity

$O(\max(n, m))$

We process each character of both strings at most once.

Reversing and joining the result also takes $O(\max(n, m))$, so the total time complexity remains:

$O(\max(n, m))$

Space Complexity

$O(\max(n, m))$

The output string can have at most $\max(n, m) + 1$ characters because of a possible final carry.

For example:

```

111
+ 1
----
1000

```

So we need linear extra space to store the result.

5. Alternative Approaches

Alternative 1: Convert to integers directly

In Python, you could do:

```

class Solution:
    def addBinary(self, a: str, b: str) -> str:
        return bin(int(a, 2) + int(b, 2))[2:]

```

This is very short, but it is usually not the intended solution because the problem is designed to test manual binary addition. Also, for very large strings, relying on integer conversion may not be allowed in some interview settings.

Alternative 2: Pad the shorter string with zeros

You can make both strings the same length first:

```
a = a.zfill(max_len)
b = b.zfill(max_len)
```

Then iterate from right to left. This makes the logic slightly simpler, but uses extra space for padding.

Alternative 3: Use bit manipulation after conversion

Another Python-specific approach:

```
class Solution:
    def addBinary(self, a: str, b: str) -> str:
        x = int(a, 2)
        y = int(b, 2)
        return bin(x + y)[2:]
```

Again, concise but not ideal if the goal is to implement binary addition manually.

The manual carry-based approach is the best general solution because it works for arbitrarily long binary strings and directly follows the binary addition process.